

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 4.1: Motivation & the Optimal Growth Running Example

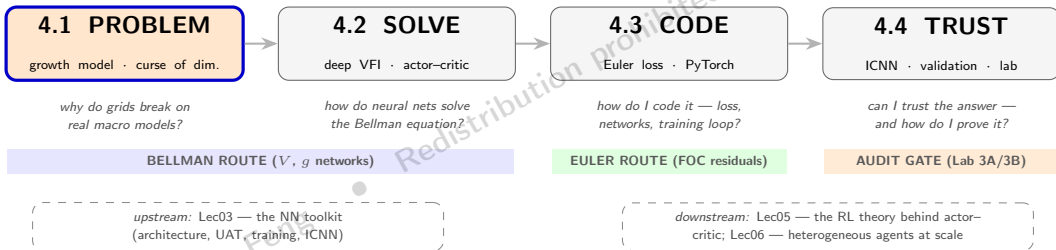
● Zhigang Feng

2026-07-07 10:05:37

Lecture 4 in One Map

One lecture, four decks, one running example: solve the optimal growth model with deep learning — pose the problem, solve it two ways, code it, and prove the answer right.

you are here ▼



Act 4.1 (here) poses the problem: the workhorse growth model and its Bellman equation, why grid methods hit the N^d wall, and the closed-form benchmark plus grid VFI (Lab 3A) that will serve as ground truth for every neural method that follows.

Motivation: Why Deep Learning for Macro Models?

The workhorse growth model — and the N^d wall every grid method hits

The Workhorse: The Optimal Growth Model

Nearly every quantitative macro model is a variation on one core structure — the optimal (neoclassical) growth model.

A planner maximizes lifetime welfare:

$$\max_{\{c_t, k_{t+1}\}} \sum_{t=0}^{\infty} \beta^t u(c_t)$$

subject to the economy's resource constraint:

$$c_t + k_{t+1} = \underbrace{f(k_t) + (1 - \delta)k_t}_{\text{resources } y_t}, \quad k_0 \text{ given}$$

Cobb–Douglas output $f(k) = Ak^\alpha$; resources = output + undepreciated capital, split between consumption c_t and next-period capital k_{t+1} .

Anatomy of the model:

Preferences	concave u , discount $\beta \in (0, 1)$
Technology	$f(k) = Ak^\alpha$, deprec. δ
State	capital k (inherited)
Control	saving $k' = g(k)$
Solve for	value $V(k)$, policy $g(k)$

Two primitives — tastes (u, β) and technology (f, δ) — plus one accounting identity generate the entire dynamic problem.

The Workhorse: Recursive (Bellman) Formulation

The infinite-horizon plan collapses to a one-step problem in a single state.

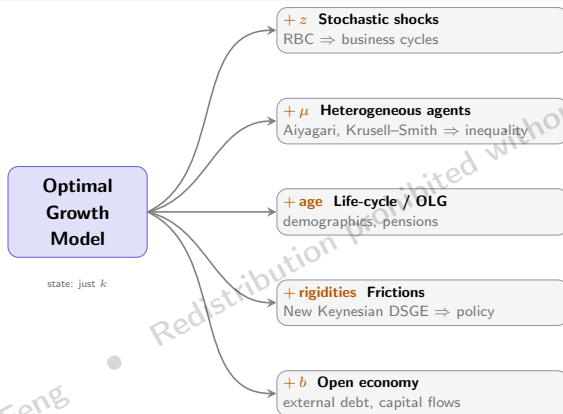
- **From sequence to recursion:** rather than choose the entire path $\{c_t, k_{t+1}\}_{t=0}^{\infty}$ at once, ask only — given capital k today, how much should we save?
- **Bellman equation** — today's value = current utility + discounted continuation value:

$$V(k) = \max_{0 \leq k' \leq y} \{ u(y - k') + \beta V(k') \}, \quad y = f(k) + (1 - \delta)k$$

- The solution is a *pair*: the value function $V(k)$ and the policy $k' = g(k)$. Consumption — and, in market versions, prices — follow from g .
- **A single state variable** k makes this cheap to solve on a grid — and that very tractability is what breaks once the model grows richer.

Running Example (Section 2) specializes this to the analytically solvable case ($A = 1$, $\delta = 1$, $u = \log$), giving the closed-form policy $c^(y) = (1 - \alpha\beta)y$ — our ground truth for every neural method.*

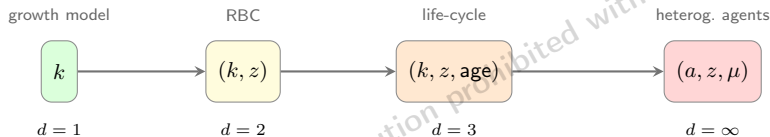
Why a Workhorse? One Model, Many Directions



Same skeleton, richer flesh — the trunk is the optimal growth model, each realistic direction a branch (cf. the extensions map in Lecture 1). Master the trunk once and every branch is within reach — but *each branch adds state variables*, and more state means “harder to compute.”

The Catch: Every Direction Adds State Variables

- Each realistic ingredient enlarges the **state vector** the solution must carry:



- A grid needs N^d points — cost **explodes** with every added dimension (Bellman's *curse of dimensionality*).
- The workhorse is trivial at $d = 1$; its research-frontier descendants quickly reach $d \geq 10$ — or carry a whole *distribution* μ (infinite-dimensional).
- This is the wall.** The rest of Topic 4.1 asks: can *deep learning* scale where grids cannot?

The Computational Frontier in Macro

- Quantitative macroeconomics solves dynamic optimization problems:

$$V(k) = \max_{k'} \left\{ u(f(k) - k') + \beta V(k') \right\}$$

- **Standard toolkit:** Value Function Iteration (VFI), Euler equation methods, perturbation
- These methods have served the profession well for decades
- But they face fundamental limitations as models grow richer:
 - More state variables (capital, bonds, housing, human capital. . .)
 - Heterogeneous agents (wealth distributions, idiosyncratic risk)
 - Aggregate uncertainty (business cycles in HA models)
- **This lecture:** How deep learning and reinforcement learning offer a fundamentally different approach

*Course arc: Lec02 T2 framed AI as **cognitive capital** (the why); Lec03 delivered NN architectures and training algorithms (the what); Lec04 closes the loop — turning those tools on the optimal-growth Bellman equation (the how).*

The Naive Approach: Grid-Based Methods

- **Value Function Iteration on a grid:**

1. Discretize the state space: $\{k_1, k_2, \dots, k_N\}$
2. For each grid point, solve $\max_{k'} \{u(f(k_i) - k') + \beta V(k')\}$
3. Iterate until convergence

- **This works well** when:

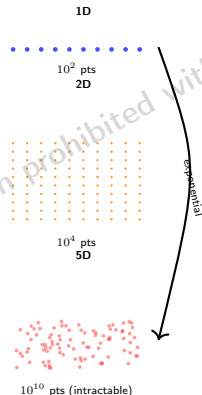
- The state space is low-dimensional (1-3 variables)
 - The functions are smooth
 - We can afford to evaluate V at every grid point
- Alternatives like Chebyshev polynomials and finite elements improve efficiency but share the same fundamental scaling problem

The Curse of Dimensionality: The N^d Explosion

A grid places N points *per dimension*, so covering d dimensions costs N^d points. With $N = 100$:

- $d = 1$: 10^2 pts (trivial)
- $d = 2$: 10^4 pts (manageable)
- $d = 5$: 10^{10} pts (infeasible)
- $d = 10$: 10^{20} pts (absurd)

Linear in dimension, but exponential in resolution — every extra state variable multiplies the cost by another factor of N .



Dots are a schematic sample, not the literal count ($N = 100$ per axis).

Why Grids Break: Real Macro Models Are High-Dimensional

- **Real macro models** routinely have $d \geq 10$ — and often far more:
 - Krusell & Smith (1998), JPE: individual k + aggregate K + shocks
 - Life-cycle: age + assets + earnings + health + $\dots$
 - HA + aggregate risk: the *whole* wealth distribution (infinite-dimensional)
- *Canonical HA benchmarks*: Aiyagari (1994), QJE; Krusell & Smith (1998), JPE.
- These are the curse-of-dimensionality test cases **Lec06** attacks with DL/RL methods; **Lec09** reuses Aiyagari as its running example.
- **Grid-based methods cannot scale to these problems** — at 10^{20} points, no amount of hardware helps.

Beyond Grids: Other Classical Limitations

- **Perturbation methods** (linearization around steady state):
 - Fast and accurate locally
 - But may lose accuracy far from steady state
 - Struggle with non-linearities, occasionally binding constraints, regime switches
- **Projection methods** (Chebyshev, finite elements):
 - Globally accurate for smooth problems
 - Require structured grids or node sets
 - Still suffer from the curse of dimensionality
- **Kinks and non-convexities:**
 - Borrowing constraints, irreversible investment, discrete choices
 - First-order conditions fail or require complex root-finding
- We need function approximators that scale gracefully with dimension and handle irregular structures

The Deep Learning Opportunity: Approximation That Scales

- **Neural networks are universal function approximators:**
 - With enough hidden units, a network can represent *any* continuous function on a compact set (Cybenko 1989; Hornik et al. 1989).
- **And they scale with dimension:**
 - Parameter count grows *polynomially* with the input dimension, not exponentially (Barron 1993).
 - A grid needs N^d points; a network needs a weight count that grows gently with d .
- **So represent the value or policy function as a network** — $V_\theta(k)$ or $g_\theta(k)$ — and *learn* the weights θ ; there is no grid to discretize.
- This is the escape from the curse of dimensionality: replace “store a value at every grid point” with “fit one smooth function that generalizes.”

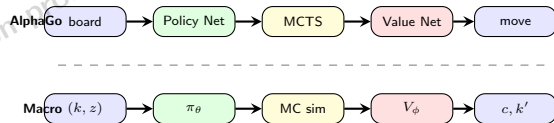
The Deep Learning Opportunity: Differentiation & Hardware

- **Automatic differentiation gives gradients for free:** Euler equations, FOCs, and Jacobians are differentiated straight from the code (PyTorch) — no hand-derived, hand-coded derivatives.
- **Why GPUs accelerate deep learning:**
 - A network's forward and backward passes are *dense matrix multiplications* (Wx); training repeats billions of multiply-adds.
 - A GPU packs *thousands* of arithmetic cores (a CPU has a handful) that apply one operation to many numbers at once (SIMD), fed by very high memory bandwidth — exactly what matrix multiplication needs.
 - The work is *parallel across the batch*: thousands of sampled states (k, z) , or a whole panel of simulated agents, run at once — so one gradient step over thousands of states costs about what a CPU spends on a handful.
- **The key insight:** recast the macro model as a *learning problem* — the policy or value function is the object to be learned, trained by stochastic gradient descent on sampled states.

The Inspiration: AlphaGo and Economic Models

Silver et al. (2016, Nature), "Mastering the Game of Go with Deep Neural Networks and Tree Search."

- **AlphaGo** (DeepMind, 2016) solved a problem with $\sim 10^{170}$ board states.
- It did *not* enumerate the state space or solve the full Bellman equation on a grid.
- It used two neural networks:
 - **Policy Net (Actor):** "Intuition" — which move to play.
 - **Value Net (Critic):** "Judgment" — who is winning.
- **Lesson for economics:** when the state space is too vast (e.g., wealth distribution in a HA model), use the same two-network approach.
- **This lecture:** apply the actor-critic idea to the optimal growth model as a proof of concept, then discuss scaling.



Same architectural pattern; different state space.

Reframing Macro as a Machine Learning Problem

- The optimal growth model can be recast as a **learning problem**:
 - The decision rule $g(k)$ is approximated by a neural network
 - The first-order conditions provide a natural **loss function**
 - Modern optimization (Adam, SGD) and automatic differentiation drive training
- **Two complementary approaches**:
 1. **Deep Value Function Iteration**: Train networks to satisfy the Bellman equation (reinforcement learning flavor)
 2. **Euler Equation Minimization**: Train a network to satisfy the FOCs (supervised learning flavor)
- Both use the same building blocks:
 - Neural network function approximation
 - Stochastic gradient descent
 - Monte Carlo sampling of state space

Running Example: The Optimal Growth Model

A closed-form benchmark and grid VFI: the ground truth the networks must beat

Optimal Growth Model: Setup

Recall the workhorse from Section 1 — here we pin down the analytically solvable special case.

- **Objective:** Maximize discounted lifetime utility:

$$\max_{\{k_{t+1}\}} \sum_{t=0}^{\infty} \beta^t u(f(k_t) - k_{t+1})$$

- **Production function:** $f(k_t) = k_t^\alpha$
- **Resource constraint:** Consumption $c_t = f(k_t) - k_{t+1}$
- **Depreciation:** $\delta = 1$ (full depreciation) in our benchmark; adjustable for general cases
- **Steady-state capital:**

$$k_{ss} = \left(\frac{1}{\alpha\beta} \right)^{\frac{1}{\alpha-1}} = (\alpha\beta)^{\frac{1}{1-\alpha}}$$

- **Why this model?** Simple enough for an analytical solution (ground truth), yet contains all the essential structure of richer models

Bellman Equation and VFI

- **Recursive formulation** (Bellman equation):

$$V(k) = \max_{k'} \left\{ u(f(k) - k') + \beta V(k') \right\}$$

- **Value Function Iteration:**

1. Start with initial guess $V^0(k)$
2. Update: $V^{n+1}(k) = \max_{k'} \{ u(f(k) - k') + \beta V^n(k') \}$
3. Repeat until $\|V^{n+1} - V^n\| < \text{tolerance}$

- **Contraction Mapping Theorem (Banach):** Under standard assumptions, the Bellman operator T is a contraction with modulus β
 - Guarantees convergence to a unique fixed point V^*
 - VFI converges *geometrically* at rate β
 - This is the theoretical foundation we build on

Stochastic Extension

- Add a productivity shock z :

$$v(y) = \max_{0 \leq c \leq y} \left\{ u(c) + \beta \mathbb{E}[v(y')] \right\}$$

- Transition: $y' = f(y - c) \cdot z$, where z is i.i.d. log-normal
- **Analytical solution** for log utility $u(c) = \ln(c)$ and Cobb-Douglas $f(k) = k^\alpha$:

$$V^*(y) = A + B \ln y \quad (\text{value function})$$

$$c^*(y) = (1 - \alpha\beta) y \quad (\text{optimal policy: consume fixed fraction})$$

- This closed-form solution serves as our **ground truth** for validating all neural network methods
- Lab 3A builds this benchmark via classical VFI with interpolation

VFI with Interpolation: The Classical Algorithm

- Since y is continuous, we use interpolation on a grid:

1. **Initialize:** Grid $\{y_1, \dots, y_N\}$, initial guess v^0
2. **Interpolate:** Create function $\hat{v}(y)$ connecting (y_i, v_i^0) pairs
3. **Bellman update:** For each y_i :

$$v_i^{\text{new}} = \max_c \left\{ \ln(c) + \beta \sum_j \hat{v}(f(y_i - c) z_j) \phi(z_j) \right\}$$

4. **Check convergence:** If $\|v^{\text{new}} - v^0\| < \text{tol}$, stop; else repeat

- This is the **benchmark** algorithm implemented in Lab 3A
- Works beautifully for 1D... but the grid makes it impractical for higher dimensions

Bridge: The Problem Is Set — Now Replace the Grid

Act 4.1 posed the problem. The optimal growth model is the workhorse of quantitative macro, its Bellman equation is the object every solver must satisfy — and grid-based VFI, exact in one dimension, collapses under the N^d explosion.

- You now hold the benchmark: a closed-form special case and grid VFI (Lab 3A) — the ground truth every neural solution in this lecture must reproduce.
- **Next (4.2):** swap the grid for parameters — approximate V and g with neural networks and train them on the Bellman equation itself, actor–critic style.

Arc: **4.1 PROBLEM** (*growth model + curse*) $\rightarrow$ **4.2 SOLVE** (*deep VFI + actor–critic*) $\rightarrow$ **4.3 CODE** (*Euler + PyTorch*) $\rightarrow$ **4.4 TRUST** (*structure + validation*).